Exponents, Logarithms, and Compounding

Core Math for Quant Research

BEGINNER FIELD GUIDE

A dense single-column reference manual for turning growth, decay, returns, logs, and compounding into usable research-system intuition.

CORE QUESTION

How does repeated percentage change become a path?

QUANT USE

Returns, CAGR, log prices, volatility drag, decay, and annualization.

SYSTEM FIT

Data -> features -> models -> signals -> portfolio growth -> diagnostics.

Table of Contents

REFERENCE

MANUAL STRUCTURE

Use the page numbers as a fast lookup map. Pages are standalone cards: definition first, then quant use, example, formula, implementation, system fit, mistakes, and related terms.

Table of Contents continued

REFERENCE

MANUAL STRUCTURE

Use the page numbers as a fast lookup map. Pages are standalone cards: definition first, then quant use, example, formula, implementation, system fit, mistakes, and related terms.

How to Use This Manual

GUIDE

READING METHOD

Treat each page like a field card. Read the meaning first, then the memory jogger, then the quant use. Only then inspect formulas, examples, and implementation notes. The goal is fast recall plus practical system placement.

RECOMMENDED STUDY LOOP

Step	Action	What you should retain
1	Scan the header and one-sentence meaning.	The concept's plain-English identity.
2	Read the market example and formula block.	How the math appears in price, return, feature, or portfolio data.
3	Check the system-fit box.	Where the idea belongs in your research stack.
4	Answer the quick recall prompt.	Whether the concept is actually retrievable without rereading.
5	Use the diagnostics pages at the end.	How to catch compounding and log-return mistakes in code.

CORE MENTAL MODEL

Exponents describe repeated multiplication. Logarithms undo exponential growth and turn multiplicative movement into additive movement. Compounding is the repeated application of returns through time. Quant systems use all three constantly.

WHAT TO INSPECT WHEN STUDYING

Panel	Question to ask
Formula	What is being multiplied, added, logged, or exponentiated?
Example	Does the example use a price, return, feature, signal, or portfolio value?
Implementation	What exact pandas/numpy operation would create the value?
Confusion	What is the common wrong interpretation?

MOBILE READING NOTE

This manual avoids parallel columns. Every page is one vertical stream: top header, definition, intuition, example, formulas, implementation, diagnostics, and related terms. Scroll straight down.

Notation Quick Reference

REFERENCE

NOTATION RULE

Symbols are kept close to quant usage. A symbol should point to an object in your data pipeline: price, return, growth factor, feature, parameter, weight, or portfolio value.

COMMON SYMBOLS

Symbol	Meaning	Quant location
P_t	Price at time t	Raw market data and price paths
r_t	Simple return or generic return	Percent change, strategy return, asset return
g_t	Growth factor, usually 1 + r_t	Compounding portfolio values
ell_t	Log return, log(P_t/P_{t-1})	Additive return aggregation
V_t	Portfolio/account value at time t	Equity curve and drawdown analysis
mu	Mean return or expected growth rate	Forecasting and performance summaries
sigma	Volatility or standard deviation	Risk, variance drag, annualization
lambda	Decay rate	Half-life, EMA, mean reversion
alpha	Smoothing parameter	EMA and exponentially weighted features
w_i	Portfolio weight for asset i	Position sizing and rebalancing
Sigma	Covariance matrix	Portfolio risk and diversification

NOTATION ANCHOR

```
simple return: r_t = P_t/P_{t-1} - 1
log return: ell_t = log(P_t/P_{t-1})
growth factor: g_t = 1 + r_t
compounded value: V_T = V_0 * product_t(g_t)
```

When stuck, ask whether the object is a price level, a return, a growth factor, or a logged value.

CONFUSION TO AVOID

Do not treat returns, log returns, price levels, and growth factors as interchangeable. They are connected by formulas, but they live in different units.

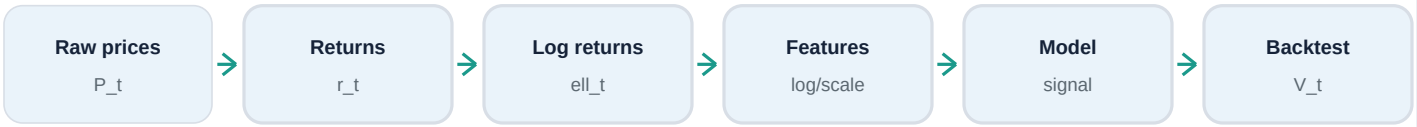
Research Pipeline Map

WORKFLOW

BIG PICTURE

Exponents, logs, and compounding are not isolated math tricks. They appear whenever a research system transforms raw prices into returns, features, signals, portfolio growth, and validation diagnostics.

PROCESS FLOW DIAGRAM



1. Logs make multiplicative changes additive.
2. Exponents convert log growth back into levels.
3. Compounding tests if a signal survives sequence and volatility.

PIPELINE CHECKPOINTS

Stage	Math object	Failure to detect
Data cleaning	positive prices and valid denominators	bad logs from zero or negative values
Feature engineering	log, log1p, exp, power, z-score	leakage, unstable scaling, distorted rank
Modeling	transformed explanatory variables	coefficients interpreted in wrong units
Backtest	growth factors and cumulative product	adding simple returns instead of compounding
Evaluation	CAGR, drawdown, log equity slope	high average return but poor path growth

RESEARCH SYSTEM CONNECTION

Your system should store raw levels, returns, log returns, and cumulative growth separately. That makes audits easier and prevents unit mistakes from silently contaminating features and backtests.

Exponents as Repeated Multiplication

BEGINNER

ONE-SENTENCE MEANING

An exponent tells you how many times a base is multiplied by itself or scaled through repeated growth. Memory jogger: Exponent means repeat the multiplier, not repeat addition.

WHY IT MATTERS IN QUANT RESEARCH

Compounding, leverage, discounting, volatility scaling, and exponential weighting all rely on repeated multiplication.

SIMPLE MARKET/TRADING EXAMPLE

If capital grows by 1.01 each day for 20 days, the ending multiplier is 1.01^20, not 1 + 20*0.01 unless you are making a simple approximation.

FORMULA / INTUITION

```
a^n = a * a * ... * a (n times)
(1 + r)^T = repeated growth factor over T periods
```

In quant systems, the base is often a growth factor and the exponent is a number of periods.

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use numpy power operations for vectorized calculations: growth = np.power(1 + r, T). Avoid Python loops when applying the same exponent to arrays.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Use exponents when you convert a per-period rate into a multi-period value or simulate repeated reinvestment.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not read 1.02^10 as 1.20. Compounding creates curvature because each period grows the previous period, not the starting value. Related terms: growth factor, compounding, annualization, exponential growth Quick recall: If a strategy earns 2 percent for 10 periods, what expression represents reinvested growth?

VALIDATION CHECKLIST

Validation checklist for Exponents as Repeated Multiplication: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Base and Exponent Roles

BEGINNER

ONE-SENTENCE MEANING

The base is the thing being repeatedly multiplied; the exponent controls how many repeated multiplications or scale steps occur. Memory jogger: Base = object. Exponent = repetition or time.

WHY IT MATTERS IN QUANT RESEARCH

Separating base from exponent prevents bad annualization and horizon conversion in performance summaries.

SIMPLE MARKET/TRADING EXAMPLE

A monthly growth factor of 1.015 compounded for 12 months is base 1.015 with exponent 12.

FORMULA / INTUITION

```
base^exponent = object being multiplied ^ number of steps
monthly to annual: (1 + r_month)^12 - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Name variables explicitly: growth_factor_monthly, periods_per_year, annual_return. Clear names prevent mixing a rate with a multiplier.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Belongs in return aggregation, horizon conversion, scenario analysis, and signal decay modules.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: The exponent is not always time, but in quant it often behaves like time, lag count, or compounding frequency. Related terms: growth factor, time horizon, compounding frequency, annualization Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Base and Exponent Roles: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Powers of Ten and Scale

BEGINNER

ONE-SENTENCE MEANING

Powers of ten express orders of magnitude and help compare small returns, large notionals, and tiny probabilities. Memory jogger: Each power of ten shifts the decimal point.

WHY IT MATTERS IN QUANT RESEARCH

Market data often spans very different scales: basis points, percentages, dollars, millions, and billions.

SIMPLE MARKET/TRADING EXAMPLE

A one basis point move is 0.0001 in decimal return units. A million-dollar position times 0.0001 is a 100 dollar move.

FORMULA / INTUITION

```
10^3 = 1,000
10^-4 = 0.0001 = 1 basis point
notional P&L approx = notional * return
```

SCALE TRANSLATION TABLE

Expression	Decimal	Market phrase
10^-4	0.0001	one basis point
10^-2	0.01	one percent
10^6	1,000,000	one million
10^9	1,000,000,000	one billion

PYTHON / SYSTEM IMPLEMENTATION IDEA

Store returns in decimal form, not percent strings. A 2 percent return should be 0.02 in the research database.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Useful for data validation, P&L checks, feature scaling, and sanity checks on reported metrics.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not mix basis points, percentages, and decimal returns in the same column without metadata. Related terms: basis points, decimal return, normalization, feature scaling Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Powers of Ten and Scale: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Negative Exponents

BEGINNER

ONE-SENTENCE MEANING

A negative exponent means reciprocal scaling, such as dividing by repeated powers of the base. Memory jogger: Negative exponent flips the multiplier into the denominator.

WHY IT MATTERS IN QUANT RESEARCH

Discounting and inverse scale relationships often use negative exponents.

SIMPLE MARKET/TRADING EXAMPLE

A present-value discount factor can be written as (1 + r)^-T, which shrinks future cash flow back to today.

FORMULA / INTUITION

```
a^-n = 1 / a^n
PV = FV / (1 + r)^T = FV * (1 + r)^-T
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use negative powers to compute discount factors directly: df = np.power(1 + rate, -periods).

WHERE THIS FITS IN THE RESEARCH SYSTEM

Appears in valuation, risk-neutral discounting, carry calculations, and scenario present-value modules.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A negative exponent is not a negative value. It creates a reciprocal. The output can still be positive. Related terms: reciprocal, discount factor, present value, compounding Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Negative Exponents: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Fractional Exponents and Roots

BEGINNER

ONE-SENTENCE MEANING

A fractional exponent represents a root or partial-period growth conversion. Memory jogger: Power of 1/n means nth root.

WHY IT MATTERS IN QUANT RESEARCH

You use fractional exponents when converting long-horizon growth into per-period growth, such as CAGR.

SIMPLE MARKET/TRADING EXAMPLE

If capital doubles over 4 years, annual growth is 2^(1/4) - 1, not 2/4.

FORMULA / INTUITION

```
a^(1/n) = nth root of a
per_period_growth = total_growth^(1/T) - 1
```

This is the mathematical backbone of CAGR and geometric average return.

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.power(total_growth, 1 / periods) - 1. Validate periods > 0 and total_growth > 0.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Used in performance reporting, annualization, horizon normalization, and comparing strategies with different test lengths.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not divide total return by years when the return was reinvested. Use the geometric conversion. Related terms: CAGR, geometric mean, roots, annualized return Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Fractional Exponents and Roots: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Exponent Laws

BEGINNER

ONE-SENTENCE MEANING

Exponent laws are shortcuts for combining powers with the same base or converting powers of powers. Memory jogger: Same base lets exponents add; power of a power lets them multiply.

WHY IT MATTERS IN QUANT RESEARCH

Exponent rules simplify compounding formulas and help verify whether code is aggregating growth correctly.

SIMPLE MARKET/TRADING EXAMPLE

Compounding for 12 months and then 12 more months gives (1+r)^12 * (1+r)^12 = (1+r)^24.

FORMULA / INTUITION

```
a^m * a^n = a^(m+n)
(a^m)^n = a^(m*n)
a^m / a^n = a^(m-n)
```

EXPONENT LAW DIAGNOSTICS

Pattern	Correct simplification	Quant interpretation
same base multiplied	add exponents	extend horizon
power of power	multiply exponents	frequency conversion
same base divided	subtract exponents	relative growth over subperiod

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use formulas to write unit tests: compounding a daily growth factor for 252 days then another 252 days should equal compounding for 504 days.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Supports formula simplification, test cases, annualization checks, and backtest accounting validation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Exponent rules only work this way when bases match. You cannot add exponents for different bases. Related terms: compounding, growth factor, horizon conversion, algebraic simplification Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Exponent Laws: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Exponential Growth

BEGINNER

ONE-SENTENCE MEANING

Exponential growth occurs when a quantity grows by a constant proportion per period. Memory jogger: Same percent every period means bigger dollar changes over time.

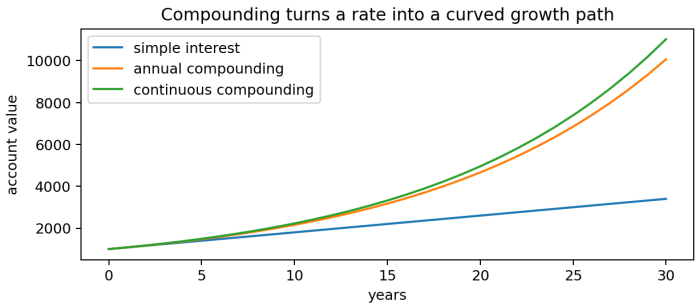
WHY IT MATTERS IN QUANT RESEARCH

Equity curves, account growth, reinvested returns, and compounded inflation can all follow exponential logic.

SIMPLE MARKET/TRADING EXAMPLE

A strategy with a stable positive average log return produces an equity curve that trends upward exponentially in level terms.

SYNTHETIC GROWTH CURVES



The curved paths come from applying growth to the new value each period.

FORMULA / INTUITION

```
V_t = V_0 * (1 + r)^t
continuous form: V_t = V_0 * exp(g*t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

For a constant rate, use value = initial * np.exp(g * t) or initial * np.power(1 + r, t).

WHERE THIS FITS IN THE RESEARCH SYSTEM

Used in scenario projections, expected-growth summaries, and sanity checks on equity curve shape.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A straight line on a log scale can correspond to exponential growth on a normal price scale. Related terms: compound growth, log scale, CAGR, equity curve Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Exponential Growth: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Exponential Decay

BEGINNER

ONE-SENTENCE MEANING

Exponential decay occurs when something shrinks by a constant proportion each period. Memory jogger: Each period keeps a fraction of what remains.

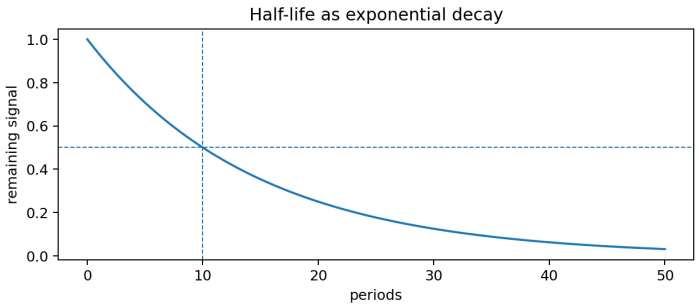
WHY IT MATTERS IN QUANT RESEARCH

Signal decay, mean reversion, EMA weights, and half-life estimates use exponential decay.

SIMPLE MARKET/TRADING EXAMPLE

A signal with half-life 5 days has roughly half of its initial information value after 5 days, one quarter after 10 days, and so on.

HALF-LIFE DECAY CURVE



The curve falls quickly at first, then slowly as less signal remains.

FORMULA / INTUITION

```
remaining_t = initial * exp(-lambda*t)
half_life = log(2) / lambda
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use decay = np.exp(-lambda_ * lag) for lag features and signal freshness weights.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Belongs in signal aging, feature decay, alpha half-life research, and execution timing.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Decay is proportional, not subtracting the same amount each period. Related terms: half-life, EMA, mean reversion, lambda Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Exponential Decay: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Growth Factor

BEGINNER

ONE-SENTENCE MEANING

A growth factor is one plus a return; it is the multiplier that moves value from one period to the next. Memory jogger: Return tells change; growth factor performs the multiplication.

WHY IT MATTERS IN QUANT RESEARCH

Backtests compound growth factors, not raw percentage strings.

SIMPLE MARKET/TRADING EXAMPLE

A 3 percent return has growth factor 1.03. A -3 percent return has growth factor 0.97.

FORMULA / INTUITION

```
g_t = 1 + r_t
V_t = V_{t-1} * g_t
```

RETURN TO GROWTH FACTOR

Return r_t	Growth factor g_t	Meaning
0.02	1.02	gain 2 percent
0.00	1.00	flat
-0.05	0.95	lose 5 percent
-1.00	0.00	total loss

PYTHON / SYSTEM IMPLEMENTATION IDEA

In pandas, compute growth = 1 + returns and equity = initial * growth.cumprod().

WHERE THIS FITS IN THE RESEARCH SYSTEM

Central object in portfolio simulation, backtesting, drawdown, and compounding logic.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not cumulative-sum simple returns to create an equity curve. Use cumulative product of growth factors. Related terms: return, compounding, cumulative product, equity curve Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Growth Factor: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Compound Growth

BEGINNER

ONE-SENTENCE MEANING

Compound growth means each period return applies to the current value, not just the original starting value. Memory jogger: Growth earns growth on prior growth.

WHY IT MATTERS IN QUANT RESEARCH

Any realistic equity curve with reinvestment is compounded; this is why return sequence and drawdowns matter.

SIMPLE MARKET/TRADING EXAMPLE

A 50 percent loss followed by a 50 percent gain leaves you at 0.5 * 1.5 = 0.75, a 25 percent loss overall.

FORMULA / INTUITION

```
V_T = V_0 * product_{t=1..T}(1 + r_t)
total_return = product(1 + r_t) - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use cumprod, not sum: equity_curve = start_value * (1 + returns).cumprod().

WHERE THIS FITS IN THE RESEARCH SYSTEM

The backtest accounting core: every strategy equity curve is a compounded sequence of growth factors.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Positive and negative equal-sized simple returns do not cancel after compounding. Related terms: growth factor, cumulative product, volatility drag, path dependence Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Compound Growth: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Simple Interest vs Compound Interest

BEGINNER

ONE-SENTENCE MEANING

Simple interest grows from the original principal; compound interest grows from the updated balance. Memory jogger: Simple adds; compound multiplies.

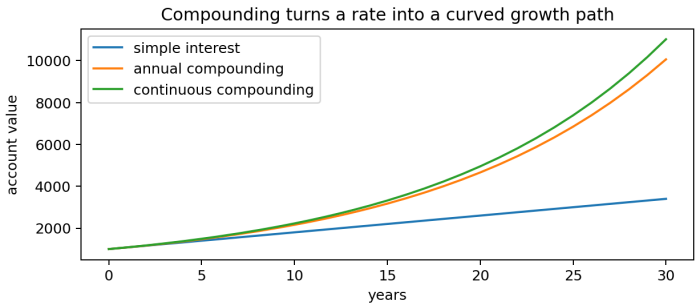
WHY IT MATTERS IN QUANT RESEARCH

Simple approximations can be useful for tiny short-horizon moves, but backtests need compounding for realistic account paths.

SIMPLE MARKET/TRADING EXAMPLE

At 10 percent for 3 years, simple interest gives 1.30x. Annual compounding gives 1.10^3 = 1.331x.

SIMPLE VS COMPOUNDED GROWTH



The gap widens as rate and time horizon increase.

FORMULA / INTUITION

```
simple: V_T = V_0 * (1 + r*T)
compound: V_T = V_0 * (1 + r)^T
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use simple formulas only for approximation and explanatory checks. Use compounded formulas for portfolio values.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Useful in reports where you must explain why CAGR and total return are not arithmetic averages.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not use simple interest logic to annualize a multi-year backtest return. Related terms: CAGR, annualization, total return, growth factor
Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Simple Interest vs Compound Interest: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Cumulative Product

BEGINNER

ONE-SENTENCE MEANING

Cumulative product multiplies a sequence step by step, preserving the path of compounded growth. Memory jogger: Cumprod is the backtest equity-curve operator.

WHY IT MATTERS IN QUANT RESEARCH

It turns a return series into an equity curve and exposes path-dependent drawdown behavior.

SIMPLE MARKET/TRADING EXAMPLE

Growth factors [1.02, 0.98, 1.01] produce cumulative growth [1.02, 0.9996, 1.009596].

FORMULA / INTUITION

```
cum_growth_t = product_{i=1..t}(1 + r_i)
equity_t = equity_0 * cum_growth_t
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

pandas: growth = 1 + returns; equity = start * growth.cumprod(). Check for returns <= -1.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Lives in backtesting, portfolio accounting, risk reporting, and equity curve generation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Cumulative product is not cumulative sum. Cumprod is multiplicative; cumsum is additive. Related terms: growth factor, compound return, equity curve, drawdown Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Cumulative Product: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Geometric Return

BEGINNER

ONE-SENTENCE MEANING

A geometric return is the constant per-period return that would produce the same total compounded growth. Memory jogger: Geometric return is the steady compound rate.

WHY IT MATTERS IN QUANT RESEARCH

It compares strategies over different horizons more honestly than the arithmetic average when compounding matters.

SIMPLE MARKET/TRADING EXAMPLE

If a strategy grows from 100 to 121 over 2 years, the geometric annual return is sqrt(1.21)-1 = 10 percent.

FORMULA / INTUITION

```
geometric_return = (V_T/V_0)^(1/T) - 1
also: exp(mean(log(1+r_t))) - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Compute total_growth = equity.iloc[-1] / equity.iloc[0]; geo = total_growth**(1/periods) - 1.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Used in performance summaries, strategy comparison dashboards, and benchmark-normalized reports.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Arithmetic average return can overstate actual compounded growth when volatility is high. Related terms: CAGR, log return, total return, volatility drag Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Geometric Return: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

CAGR

BEGINNER

ONE-SENTENCE MEANING

CAGR is the annualized geometric growth rate from starting value to ending value. Memory jogger: CAGR answers: what steady yearly return got me here?

WHY IT MATTERS IN QUANT RESEARCH

CAGR is a standard performance metric for comparing backtests with different lengths.

SIMPLE MARKET/TRADING EXAMPLE

An equity curve rising from 100,000 to 160,000 over 4 years has CAGR = (1.6)^(1/4)-1.

FORMULA / INTUITION

```
CAGR = (ending_value / starting_value)^(1 / years) - 1
```

CAGR INPUTS

Input	Meaning	Common mistake
ending value	final equity level	using total profit only
starting value	initial capital	forgetting deposits/withdrawals
years	elapsed time in years	using trade count as time
output	annual geometric growth	calling it average monthly return

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use actual elapsed time for years. If using daily data, years = len(trading_days) / 252 as an approximation.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Performance reporting layer, strategy comparison, investor-style summary dashboards.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: CAGR ignores path risk. Pair it with max drawdown, volatility, Sharpe, and turnover. Related terms: geometric return, total return, annualization, drawdown Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for CAGR: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Annualization

BEGINNER

ONE-SENTENCE MEANING

Annualization converts a shorter-period return or statistic into a yearly scale. Memory jogger: Annualization changes horizon, not strategy quality.

WHY IT MATTERS IN QUANT RESEARCH

Daily, weekly, and monthly results need common units before comparing strategies.

SIMPLE MARKET/TRADING EXAMPLE

A daily log return mean can be annualized by multiplying by about 252; a simple daily return needs compounding logic.

FORMULA / INTUITION

```
simple constant daily return: (1 + r_daily)^252 - 1
mean daily log return: annual_log = 252 * mean_log_daily
annual simple from log: exp(annual_log) - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use different rules for means, volatilities, and compounded returns. Annual vol usually scales by sqrt(252), not 252.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Metrics layer: CAGR, annual return, annual volatility, Sharpe ratio inputs, report cards.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not multiply daily simple returns by 252 unless explicitly making a small-return approximation. Related terms: CAGR, volatility scaling, log returns, compounding frequency Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Annualization: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Continuous Compounding

BEGINNER

ONE-SENTENCE MEANING

Continuous compounding is the limit where growth is compounded infinitely often and represented with exp. Memory jogger: Continuous compounding uses e as the growth machine.

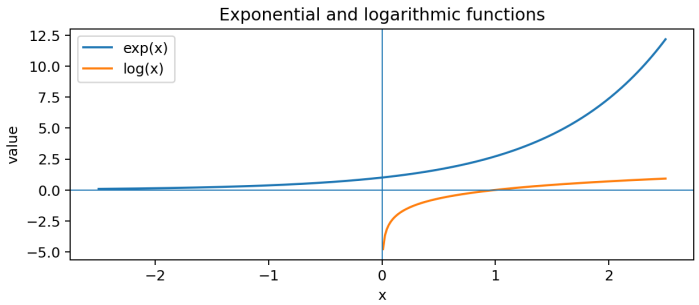
WHY IT MATTERS IN QUANT RESEARCH

Continuous compounding connects log returns, pricing formulas, and many continuous-time finance models.

SIMPLE MARKET/TRADING EXAMPLE

A continuously compounded rate g for T years gives final value V_T = V_0 * exp(gT).

EXP AND LOG CONNECTION



exp turns log growth into a level; log turns level growth into additive growth.

FORMULA / INTUITION

```
V_T = V_0 * exp(g*T)
g = log(V_T/V_0) / T
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.exp(log_growth) to convert log cumulative returns back into growth factors.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Used in log-return aggregation, option-pricing foundations, and continuous-time modeling intuition.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Continuous compounding is not magic free return. It is a mathematical convention for representing growth. Related terms: Euler number e, log return, exp, continuous rate Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Continuous Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Euler Number e

BEGINNER

ONE-SENTENCE MEANING

e is the natural base for continuous growth and decay. Memory jogger: e is the base that makes exp and natural log work cleanly.

WHY IT MATTERS IN QUANT RESEARCH

Natural logs and exp are standard in return math because they simplify compounding and aggregation.

SIMPLE MARKET/TRADING EXAMPLE

If cumulative log return is 0.20, total growth factor is exp(0.20) about 1.221, or roughly 22.1 percent total return.

FORMULA / INTUITION

```
e approx 2.71828
exp(x) = e^x
log(exp(x)) = x
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.exp and np.log rather than manually approximating e. For tiny values, prefer np.expm1 and np.log1p.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Core numeric transformation in feature engineering, return aggregation, and growth conversion.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: e is not a return. It is the base of the natural exponential function. Related terms: natural log, exp, continuous compounding, log return
Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Euler Number e: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Logarithms

BEGINNER

ONE-SENTENCE MEANING

A logarithm tells you what exponent is needed to produce a value from a chosen base. Memory jogger: Log asks: what power got me here?

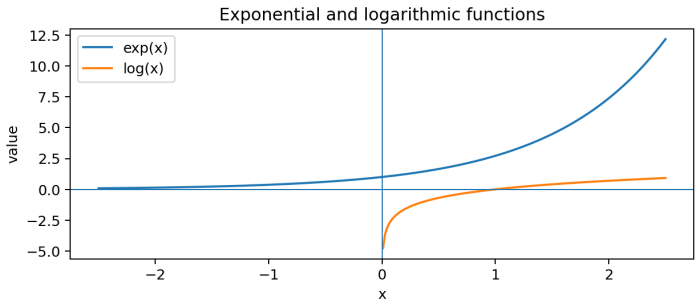
WHY IT MATTERS IN QUANT RESEARCH

Logs convert multiplicative ratios into additive values, making compounded returns easier to analyze.

SIMPLE MARKET/TRADING EXAMPLE

If a price ratio is 1.10, log(1.10) gives the continuously compounded return that maps 1.0 to 1.10.

LOG AS INVERSE OF EXPONENT



The log curve is defined only for positive inputs.

FORMULA / INTUITION

```
if b^x = y, then log_b(y) = x
natural log: log(y) means log_e(y) in most quant code
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

In Python, np.log means natural log. Use np.log10 only when you deliberately want base-10 scale.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Used in log returns, transformations of skewed features, likelihoods, and growth-rate conversions.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: You cannot take the normal log of zero or negative values. Validate inputs first. Related terms: natural log, log return, inverse function, exponential function Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Logarithms: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Natural Log

BEGINNER

ONE-SENTENCE MEANING

The natural log is the logarithm with base e and is the standard log used in continuous growth math. Memory jogger: Natural log is the log of compounding.

WHY IT MATTERS IN QUANT RESEARCH

Most return aggregation, likelihood math, and exponential decay formulas use natural logs.

SIMPLE MARKET/TRADING EXAMPLE

Daily log return is usually computed as log(P_t / P_{t-1}), using the natural log.

FORMULA / INTUITION

```
ln(x) = log_e(x)
ln(P_t/P_{t-1}) = ln(P_t) - ln(P_{t-1})
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.log for natural log. Many libraries write natural log as log, not ln.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Feature engineering, return calculation, cumulative log-return accounting, and model diagnostics.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not assume log means base 10 in Python or finance code. It usually means natural log. Related terms: log returns, exp, continuous compounding, log-price series Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Natural Log: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log Base Change

BEGINNER

ONE-SENTENCE MEANING

Changing log base rescales the answer but preserves the ordering of positive values. Memory jogger: Different log bases tell the same story in different units.

WHY IT MATTERS IN QUANT RESEARCH

Base choice rarely matters for ranking features, but it matters for interpreting magnitudes.

SIMPLE MARKET/TRADING EXAMPLE

log10 market cap is easier to read by decimal scale, while natural log market cap is usually better for mathematical modeling.

FORMULA / INTUITION

```
log_b(x) = log_k(x) / log_k(b)
log10(x) = log(x) / log(10)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.log for modeling and np.log10 for human scale displays. Do not mix bases in one feature store without naming them.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Feature metadata, dashboards, reporting, and cross-source data consistency.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Changing base changes coefficient interpretation in linear models because the feature scale changes. Related terms: feature scaling, log transform, coefficient interpretation, natural log Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log Base Change: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log as Inverse of Exponent

BEGINNER

ONE-SENTENCE MEANING

Logarithms undo exponentials, and exponentials undo logarithms. Memory jogger: log and exp are inverse buttons.

WHY IT MATTERS IN QUANT RESEARCH

This lets you move between log-return space and price/growth-factor space cleanly.

SIMPLE MARKET/TRADING EXAMPLE

If cumulative log return is sum_l = 0.15, then growth factor is exp(0.15).

FORMULA / INTUITION

```
log(exp(x)) = x
exp(log(x)) = x, for x > 0
growth = exp(cumulative_log_return)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Round-trip test your transformations: x -> np.log(x) -> np.exp(log_x) should recover x for positive x within floating-point tolerance.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Unit tests for feature transforms, return conversion, and model output interpretation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: exp(log(x)) only works for positive x. Logs are not defined for nonpositive inputs without special transformations. Related terms: natural log, exp, log transform, inverse functions Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log as Inverse of Exponent: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log Returns

BEGINNER

ONE-SENTENCE MEANING

A log return is the natural log of a price ratio, turning multiplicative price movement into an additive return. Memory jogger: Log return is the return that adds through time.

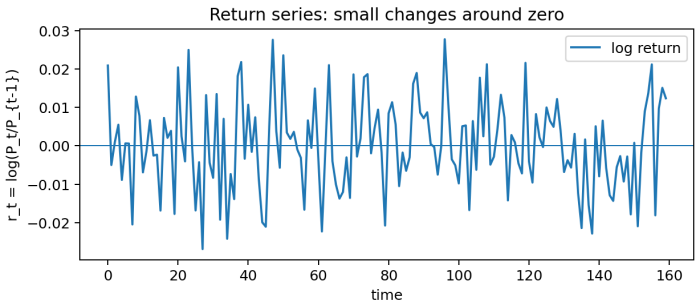
WHY IT MATTERS IN QUANT RESEARCH

Log returns simplify multi-period aggregation and connect directly to continuous compounding.

SIMPLE MARKET/TRADING EXAMPLE

If price moves from 100 to 105, log return is log(105/100), about 0.0488.

SYNTHETIC LOG RETURN SERIES



Returns fluctuate around zero while price levels compound through time.

FORMULA / INTUITION

```
ell_t = log(P_t / P_{t-1})
ell_t = log(P_t) - log(P_{t-1})
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

pandas: log_returns = np.log(price).diff(). Drop or handle the first NaN explicitly.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Return engine, feature creation, statistical modeling, volatility estimation, and cumulative growth conversion.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A log return is not exactly the same as a simple percent return, especially for large moves. Related terms: simple return, continuous compounding, log price, cumulative log return Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log Returns: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Arithmetic Return vs Log Return

BEGINNER

ONE-SENTENCE MEANING

Arithmetic return measures percent change; log return measures continuous compounded change. Memory jogger: Simple return compounds by product; log return aggregates by sum.

WHY IT MATTERS IN QUANT RESEARCH

Choosing the wrong return type can break aggregation, annualization, and feature interpretation.

SIMPLE MARKET/TRADING EXAMPLE

A +50 percent return and -50 percent return sum to zero arithmetically, but compound to a 25 percent loss.

FORMULA / INTUITION

```
simple: r_t = P_t/P_{t-1} - 1
log: ell_t = log(P_t/P_{t-1})
convert: ell_t = log(1 + r_t)
convert back: r_t = exp(ell_t) - 1
```

RETURN COMPARISON

Feature	Arithmetic return	Log return
Aggregation	compound product	sum across time
Range	cannot go below -100 percent	can approach -infinity
Best for	actual portfolio growth factors	time aggregation and modeling
Conversion	r = exp(ell)-1	ell = log(1+r)

PYTHON / SYSTEM IMPLEMENTATION IDEA

Store both when possible: simple returns for accounting, log returns for aggregation/modeling.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Return preprocessing, backtest accounting, metrics calculation, and model feature construction.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not use log returns directly as growth factors. Convert with exp before compounding level values. Related terms: percent change, log1p, expm1, cumulative return Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Arithmetic Return vs Log Return: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Additivity of Logs

BEGINNER

ONE-SENTENCE MEANING

Logs turn products into sums, which is why log returns add across time. Memory jogger: Log of product equals sum of logs.

WHY IT MATTERS IN QUANT RESEARCH

This is the clean algebra behind cumulative log return, log-likelihoods, and compounded performance math.

SIMPLE MARKET/TRADING EXAMPLE

The log of total growth over three days equals the sum of the three daily log growth factors.

FORMULA / INTUITION

```
log(a*b) = log(a) + log(b)
sum_t log(1+r_t) = log(product_t(1+r_t))
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use cumulative_log = np.log1p(returns).cumsum(); equity = start * np.exp(cumulative_log).

WHERE THIS FITS IN THE RESEARCH SYSTEM

Efficient return aggregation, portfolio report audits, and numerical stability in long backtests.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Logs add across multiplication, not across arbitrary addition. log(a+b) is not log(a)+log(b). Related terms: log return, cumulative product, log-likelihood, continuous compounding Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Additivity of Logs: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Cumulative Log Return

BEGINNER

ONE-SENTENCE MEANING

Cumulative log return is the sum of period log returns over a time window. Memory jogger: Add log returns, then exponentiate back.

WHY IT MATTERS IN QUANT RESEARCH

It gives a clean way to measure total compounded growth over many periods.

SIMPLE MARKET/TRADING EXAMPLE

If daily log returns sum to 0.30, total simple return is exp(0.30)-1, about 35 percent.

FORMULA / INTUITION

```
L_T = sum_{t=1..T} ell_t
total_growth = exp(L_T)
total_simple_return = exp(L_T) - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

cum_log = log_returns.cumsum(); equity = start * np.exp(cum_log).

WHERE THIS FITS IN THE RESEARCH SYSTEM

Return aggregation layer, signal evaluation windows, rolling performance, and log-equity charts.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Cumulative log return is not a dollar P&L. It is a logged growth measure. Related terms: log return, cumulative product, exp, total return
Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Cumulative Log Return: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log-Price Series

BEGINNER

ONE-SENTENCE MEANING

A log-price series is the natural log of price levels, making proportional changes look additive. Memory jogger: Differences in log price are log returns.

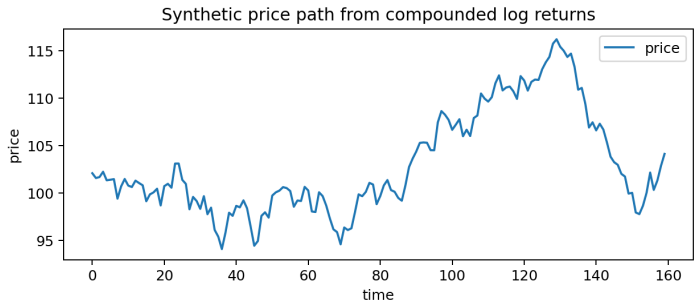
WHY IT MATTERS IN QUANT RESEARCH

Log prices are useful for trend slopes, spread modeling, cointegration intuition, and visualizing exponential growth.

SIMPLE MARKET/TRADING EXAMPLE

If log price rises in a roughly straight line, the original price is growing at a roughly constant compounded rate.

PRICE PATH FROM LOG RETURNS



A price path is the exponentiated cumulative sum of log returns.

FORMULA / INTUITION

```
x_t = log(P_t)
log return = x_t - x_{t-1}
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Create log_price = np.log(price). Then log_return = log_price.diff().

WHERE THIS FITS IN THE RESEARCH SYSTEM

Time-series preprocessing, return computation, trend analysis, and model input generation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not log adjusted and unadjusted prices inconsistently. Corporate-action handling must happen before return calculation. Related terms: log return, adjusted price, time series, trend slope Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log-Price Series: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Converting Log Returns Back to Prices

BEGINNER

ONE-SENTENCE MEANING

Exponentiating cumulative log returns converts additive log growth back into a price or equity level. Memory jogger: Sum logs, exp back to level.

WHY IT MATTERS IN QUANT RESEARCH

Models often operate in return or log-return space, but backtests and dashboards need equity curves in level space.

SIMPLE MARKET/TRADING EXAMPLE

Starting at 100, if cumulative log return is 0.10, reconstructed value is 100*exp(0.10).

FORMULA / INTUITION

```
P_t = P_0 * exp(sum_{i=1..t} ell_i)
equity_t = equity_0 * exp(cumulative_log_return_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

price_rebuilt = initial_price * np.exp(log_returns.cumsum()). Compare to original adjusted price as a QA test.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Prediction post-processing, synthetic price reconstruction, backtest reporting, and audit checks.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not add cumulative log return directly to initial price. It is a log growth measure, not a price increment. Related terms: exp, cumulative log return, synthetic price, growth factor Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Converting Log Returns Back to Prices: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

log1p

BEGINNER

ONE-SENTENCE MEANING

log1p(x) computes log(1+x) accurately, especially when x is very small. Memory jogger: Use log1p for return-to-log-return conversion.

WHY IT MATTERS IN QUANT RESEARCH

Daily returns are often tiny, so numerical accuracy matters in long cumulative calculations.

SIMPLE MARKET/TRADING EXAMPLE

For a daily simple return r_t, log return is log1p(r_t), not log(r_t).

FORMULA / INTUITION

```
ell_t = log(1 + r_t)
stable implementation: ell_t = log1p(r_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.log1p(simple_returns). Validate returns > -1 before logging.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Return preprocessing, data pipelines, numerical-stability checks, and backtest conversion utilities.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: log1p(r) means log(1+r). It does not mean log(r)+1. Related terms: log return, numerical stability, expm1, simple return Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for log1p: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

expm1

BEGINNER

ONE-SENTENCE MEANING

expm1(x) computes exp(x)-1 accurately, especially when x is very small. Memory jogger: Use expm1 for log-return-to-simple-return conversion.

WHY IT MATTERS IN QUANT RESEARCH

Small log returns can lose precision if converted with exp(x)-1 directly in some numeric contexts.

SIMPLE MARKET/TRADING EXAMPLE

A log return of 0.0002 converts to a simple return with expm1(0.0002).

FORMULA / INTUITION

```
r_t = exp(ell_t) - 1
stable implementation: r_t = expm1(ell_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.expm1(log_returns) when converting model outputs or log-return series back to simple returns.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Post-processing, signal-to-return conversion, backtest accounting, and numerical QA utilities.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: expm1 returns a simple return, not a growth factor. Add 1 if you need a multiplier. Related terms: log1p, log return, simple return, numerical stability Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for expm1: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Small-Return Approximation

BEGINNER

ONE-SENTENCE MEANING

For very small returns, log(1+r) is approximately equal to r. Memory jogger: Tiny returns make simple and log returns nearly match.

WHY IT MATTERS IN QUANT RESEARCH

This approximation explains why daily simple and log returns often look similar, while large returns diverge.

SIMPLE MARKET/TRADING EXAMPLE

A 0.1 percent return has log return about 0.0009995, almost the same as 0.001.

FORMULA / INTUITION

```
log(1 + r) approx r when |r| is small
more precise: log(1+r) approx r - r^2/2
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Approximation is acceptable for intuition, but production code should calculate the exact transform.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Used in mental checks, debugging, and explaining why daily-return models often behave similarly under either return definition.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: The approximation fails for large gains/losses, leveraged products, crypto spikes, and crash periods. Related terms: Taylor approximation, log1p, arithmetic return, volatility drag Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Small-Return Approximation: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Volatility Drag

BEGINNER

ONE-SENTENCE MEANING

Volatility drag is the reduction in compounded growth caused by variance in returns. Memory jogger: Choppy paths need more return just to end in the same place.

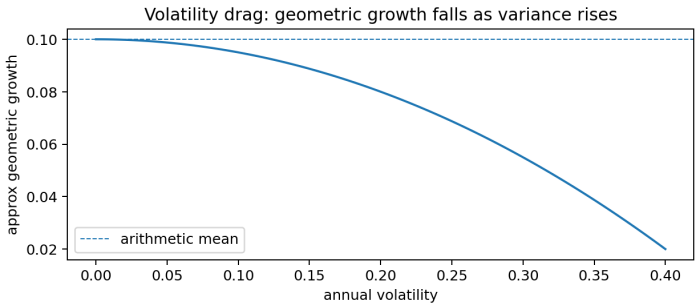
WHY IT MATTERS IN QUANT RESEARCH

It explains why average return can look good while geometric growth is weak.

SIMPLE MARKET/TRADING EXAMPLE

A +20 percent return followed by -20 percent gives 1.2*0.8 = 0.96, a 4 percent loss despite average return of zero.

VOLATILITY DRAG CURVE



For a fixed arithmetic mean, higher volatility lowers expected compound growth.

FORMULA / INTUITION

```
approx geometric growth = arithmetic mean - 0.5 * variance
log(1+r) approx r - r^2/2
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Compare arithmetic mean returns with mean log returns. A large gap is a path-risk warning.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Performance diagnostics, strategy risk review, leverage testing, and portfolio construction.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Volatility drag is not a fee. It is math from multiplicative returns. Related terms: geometric return, log return, variance, Sharpe ratio Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Volatility Drag: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Doubling Time

BEGINNER

ONE-SENTENCE MEANING

Doubling time estimates how long it takes a quantity to double under constant compounded growth. Memory jogger: More growth means shorter doubling time.

WHY IT MATTERS IN QUANT RESEARCH

Useful for sanity-checking CAGR, capital growth projections, and unrealistic backtest claims.

SIMPLE MARKET/TRADING EXAMPLE

At 10 percent continuous growth, doubling time is log(2)/0.10, about 6.93 years.

FORMULA / INTUITION

```
doubling_time = log(2) / growth_rate
discrete: T = log(2) / log(1+r)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use doubling_time = np.log(2) / np.log1p(r) for discrete periodic return r.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Report diagnostics, growth expectation checks, model sanity tests, and investor-facing explanations.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not use 2/r exactly except as a rough approximation for small rates. Related terms: half-life, CAGR, exponential growth, natural log
Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Doubling Time: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Half-Life

BEGINNER

ONE-SENTENCE MEANING

Half-life is the time it takes for a quantity or signal to decay to half its initial level. Memory jogger: Half-life is decay time in plain English.

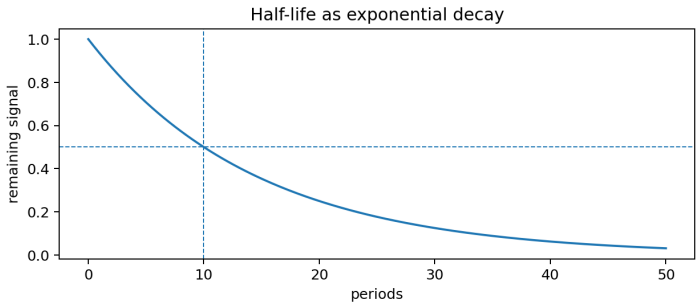
WHY IT MATTERS IN QUANT RESEARCH

Signal strength, mean reversion, and EMA responsiveness are often discussed using half-life.

SIMPLE MARKET/TRADING EXAMPLE

A mean-reversion signal with 8-day half-life should lose half of its initial effect after about 8 trading days.

HALF-LIFE VISUALIZATION



Every half-life period halves the remaining level again.

FORMULA / INTUITION

```
half_life = log(2) / lambda
lambda = log(2) / half_life
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Estimate half-life from an autoregressive coefficient: half_life = -log(2) / log(phi), when 0 < phi < 1.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Signal timing, decay weighting, model retraining cadence, and feature freshness logic.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Half-life assumes exponential decay. Not every alpha decays this cleanly. Related terms: exponential decay, mean reversion, EMA, lambda Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Half-Life: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Drawdown in Log Scale

BEGINNER

ONE-SENTENCE MEANING

Log drawdown measures the log ratio between current equity and prior peak equity. Memory jogger: Log drawdown is peak-to-current loss in log units.

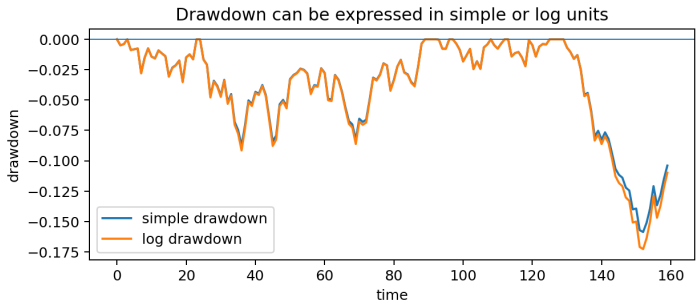
WHY IT MATTERS IN QUANT RESEARCH

It makes compounded losses additive and can simplify risk decomposition.

SIMPLE MARKET/TRADING EXAMPLE

If equity falls from 100 to 80, simple drawdown is -20 percent, while log drawdown is log(0.8).

SIMPLE VS LOG DRAWDOWN



Both show the same events, but they measure loss in different units.

FORMULA / INTUITION

```
simple_drawdown_t = V_t / max(V)_t - 1
log_drawdown_t = log(V_t / max(V)_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

peak = equity.cummax(); log_dd = np.log(equity / peak). Validate equity > 0.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Risk reporting, drawdown decomposition, equity curve diagnostics, and stress testing.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Most dashboards report simple drawdown. Label log drawdown clearly if you use it. Related terms: equity curve, cumulative log return, drawdown, risk metrics Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Drawdown in Log Scale: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Exponential Moving Average

BEGINNER

ONE-SENTENCE MEANING

An exponential moving average gives more weight to recent observations while retaining decaying memory of the past. Memory jogger: EMA remembers recent data most strongly.

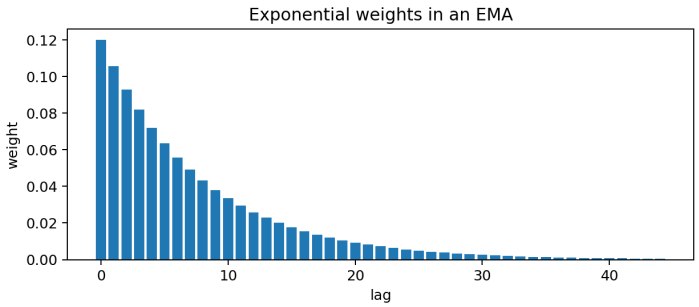
WHY IT MATTERS IN QUANT RESEARCH

EMA is used in trend features, volatility estimates, signal smoothing, and online updating.

SIMPLE MARKET/TRADING EXAMPLE

A fast EMA of price reacts quickly to recent moves; a slow EMA reacts more gradually.

EMA WEIGHTS BY LAG



Weights decline exponentially as observations become older.

FORMULA / INTUITION

```
EMA_t = alpha*x_t + (1-alpha)*EMA_{t-1}
weight_lag_k = alpha*(1-alpha)^k
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

pandas: series.ewm(alpha=alpha, adjust=False).mean(). Document alpha/span/halflife conventions.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Feature engineering, trend detection, volatility forecasting, and live streaming calculations.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Different libraries define span, alpha, and halflife differently. Store the exact parameter used. Related terms: exponential decay, half-life, smoothing, time-series feature Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Exponential Moving Average: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Exponential Weighting

BEGINNER

ONE-SENTENCE MEANING

Exponential weighting applies decaying weights so recent observations dominate the estimate. Memory jogger: Recent data counts more; old data fades.

WHY IT MATTERS IN QUANT RESEARCH

Used to estimate rolling volatility, correlations, means, and model features without hard window cutoffs.

SIMPLE MARKET/TRADING EXAMPLE

An exponentially weighted volatility estimate reacts faster to a volatility spike than a 60-day simple moving window.

FORMULA / INTUITION

```
weighted_estimate_t = sum_k w_k*x_{t-k}
w_k proportional to exp(-lambda*k)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use ewm for streaming-friendly estimates. Track warmup periods and min_periods to avoid early unstable values.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Risk engine, feature store, volatility model, covariance estimator, and online dashboards.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Exponential weighting is not the same as a fixed rolling window. There is no sharp cutoff. Related terms: EMA, decay rate, half-life, EWMA volatility Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Exponential Weighting: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Mean Reversion as Exponential Decay

BEGINNER

ONE-SENTENCE MEANING

Many mean-reverting processes can be approximated as deviations that decay exponentially toward an equilibrium. Memory jogger: The gap closes by a fraction each period.

WHY IT MATTERS IN QUANT RESEARCH

Mean-reversion strategies often estimate how fast spreads, z-scores, or residuals decay.

SIMPLE MARKET/TRADING EXAMPLE

A pairs-trading spread far above mean may be modeled as moving partially back toward the mean each day.

FORMULA / INTUITION

```
deviation_t approx deviation_0 * exp(-lambda*t)
AR(1): x_t = phi*x_{t-1} + noise
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Fit an AR(1) to deviations and convert phi to half-life only when phi is between 0 and 1.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Alpha research, spread modeling, signal holding-period design, and exit-timing logic.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A noisy series can look mean-reverting in-sample. Validate out-of-sample and across regimes. Related terms: half-life, exponential decay, AR(1), z-score Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Mean Reversion as Exponential Decay: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Discounting

BEGINNER

ONE-SENTENCE MEANING

Discounting converts a future value into present value using a growth or interest rate in reverse. Memory jogger: Discounting is compounding backward.

WHY IT MATTERS IN QUANT RESEARCH

Discounting appears in valuation, fixed income, carry, futures pricing, and capital allocation.

SIMPLE MARKET/TRADING EXAMPLE

A 105 dollar payoff one year from now discounted at 5 percent has present value 105/1.05 = 100.

FORMULA / INTUITION

```
PV = FV / (1 + r)^T
continuous: PV = FV * exp(-r*T)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use discount factors as separate columns: df = np.exp(-rate * time). This makes cash-flow valuation auditable.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Valuation engine, carry model, yield curve tools, and scenario analysis modules.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Discount rate is not automatically the expected return. It depends on pricing assumptions and risk. Related terms: present value, negative exponent, continuous compounding, yield Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Discounting: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Inflation Compounding

BEGINNER

ONE-SENTENCE MEANING

Inflation compounds price levels over time, reducing real purchasing power if nominal capital does not keep up. Memory jogger: Inflation is compounding applied to the cost of living.

WHY IT MATTERS IN QUANT RESEARCH

Real returns require subtracting or adjusting for inflation effects over time.

SIMPLE MARKET/TRADING EXAMPLE

If a portfolio earns 8 percent nominal while inflation is 3 percent, real growth is approximately 1.08/1.03 - 1, not exactly 5 percent.

FORMULA / INTUITION

```
real_growth = (1 + nominal_return) / (1 + inflation_rate) - 1
approx real return = nominal - inflation
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use exact compounding for long horizons. Approximation is acceptable only for quick small-rate intuition.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Macro features, real-return analytics, asset allocation, and long-horizon backtest interpretation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Nominal and real returns are different units. Label them clearly. Related terms: discounting, real return, growth factor, compounding
Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Inflation Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Leverage and Compounding

BEGINNER

ONE-SENTENCE MEANING

Leverage scales returns before compounding, magnifying both gains and losses through the path. Memory jogger: Leverage multiplies the ride, including the drops.

WHY IT MATTERS IN QUANT RESEARCH

Leveraged strategies can have high average returns but poor geometric growth because volatility drag increases.

SIMPLE MARKET/TRADING EXAMPLE

A 2x strategy turns +5 percent into +10 percent and -5 percent into -10 percent before costs, but the compounded path may deteriorate faster.

FORMULA / INTUITION

```
levered_return_t approx L * asset_return_t - costs_t
equity_t = equity_{t-1} * (1 + levered_return_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Apply leverage at each period before cumprod. Include borrowing, financing, margin, and rebalance assumptions.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Portfolio simulation, risk controls, stress testing, position sizing, and strategy capacity research.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Leverage is not just multiplying final CAGR. It changes the path and volatility drag. Related terms: volatility drag, drawdown, margin, growth factor Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Leverage and Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Portfolio Compounding

BEGINNER

ONE-SENTENCE MEANING

Portfolio compounding applies weighted asset returns to portfolio value across time. Memory jogger: Weights create the period return; cumprod creates the equity curve.

WHY IT MATTERS IN QUANT RESEARCH

Every portfolio backtest must convert asset returns and weights into a sequence of portfolio growth factors.

SIMPLE MARKET/TRADING EXAMPLE

A 60/40 portfolio daily return is roughly 0.6*r_stock + 0.4*r_bond before costs, then compounded over days.

FORMULA / INTUITION

```
r_p,t = sum_i w_i,t * r_i,t
V_t = V_{t-1} * (1 + r_p,t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Align weights and returns carefully. Use lagged weights if trades occur after signal formation.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Portfolio construction, backtest accounting, risk model integration, and rebalancing logic.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not use future weights to compute current returns. That creates lookahead leakage. Related terms: weights, rebalancing, growth factor, covariance Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Portfolio Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Rebalancing and Compounding

BEGINNER

ONE-SENTENCE MEANING

Rebalancing changes portfolio weights through time, which changes the compounded return path. Memory jogger: Rebalancing resets exposure; compounding records the path.

WHY IT MATTERS IN QUANT RESEARCH

Rebalance frequency affects turnover, costs, risk, and final geometric return.

SIMPLE MARKET/TRADING EXAMPLE

Monthly rebalancing may control drift better than annual rebalancing but can create more transaction costs.

FORMULA / INTUITION

```
post_return_value_i = value_i * (1 + r_i)
new_weight_i = post_return_value_i / sum_j post_return_value_j
```

REBALANCING TRADEOFFS

Frequency	Strength	Weakness
daily	tight exposure control	high turnover/costs
monthly	balanced operational choice	can miss rapid drift
annual	low turnover	large weight drift
threshold	trades only when needed	more complex logic

PYTHON / SYSTEM IMPLEMENTATION IDEA

Simulate holdings or weights explicitly. Deduct costs at rebalance events before compounding forward.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Portfolio engine, execution-cost model, turnover reports, and allocation research.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Rebalancing does not guarantee extra return. It changes exposure and path dependence. Related terms: portfolio weights, turnover, transaction costs, compounding Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Rebalancing and Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Compounding Under Volatility

BEGINNER

ONE-SENTENCE MEANING

The same average return can produce different final wealth depending on volatility and sequence. Memory jogger: Path matters because losses shrink the capital base.

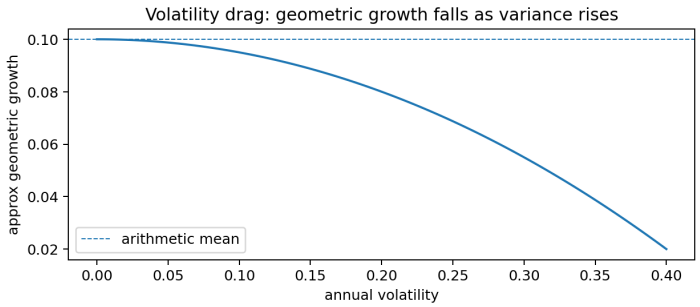
WHY IT MATTERS IN QUANT RESEARCH

Backtests must inspect equity path, drawdowns, and geometric growth, not just average return.

SIMPLE MARKET/TRADING EXAMPLE

Two strategies with identical average daily returns can have different CAGRs if one has much higher volatility.

VARIANCE REDUCES COMPOUNDED GROWTH



Higher variance lowers compound growth when arithmetic mean is held fixed.

FORMULA / INTUITION

```
geometric approx arithmetic_mean - variance/2
ending value = start * product(1+r_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Report mean return, mean log return, volatility, max drawdown, and CAGR together.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Strategy diagnostics, model ranking, risk governance, and leverage limits.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A high average return is not enough if losses are severe or clustered. Related terms: volatility drag, drawdown, geometric return, Sharpe ratio Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Compounding Under Volatility: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Compounding Frequency

BEGINNER

ONE-SENTENCE MEANING

Compounding frequency is how often a rate is applied within a period. Memory jogger: More compounding events means the rate is applied more often.

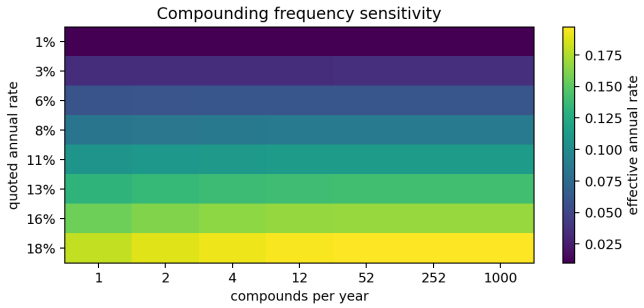
WHY IT MATTERS IN QUANT RESEARCH

Frequency matters in interest-rate calculations, effective annual rates, and comparing quoted returns.

SIMPLE MARKET/TRADING EXAMPLE

A 12 percent annual rate compounded monthly has effective annual return (1+0.12/12)^12 - 1.

COMPOUNDING FREQUENCY HEATMAP



As frequency rises, effective rate approaches the continuous compounding limit.

FORMULA / INTUITION

```
effective_rate = (1 + quoted_rate/m)^m - 1
continuous limit = exp(quoted_rate) - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Keep frequency metadata with every rate. A rate without frequency is ambiguous.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Rates, yield curves, carry models, performance reporting, and financing assumptions.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A quoted annual rate and effective annual return are not always the same. Related terms: continuous compounding, effective annual rate, APR, yield Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Compounding Frequency: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Effective Annual Rate

BEGINNER

ONE-SENTENCE MEANING

Effective annual rate is the actual one-year growth after accounting for compounding frequency. Memory jogger: Effective rate is what you actually earn over the year.

WHY IT MATTERS IN QUANT RESEARCH

It standardizes rates with different compounding conventions.

SIMPLE MARKET/TRADING EXAMPLE

A monthly-compounded 6 percent quoted rate becomes (1+0.06/12)^12 - 1.

FORMULA / INTUITION

```
EAR = (1 + nominal_rate/m)^m - 1
continuous EAR = exp(rate) - 1
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Write a utility function nominal_to_ear(rate, m). Unit test m=1 against simple annual compounding.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Rate normalization, cash management, funding cost model, and benchmark comparisons.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: APR, nominal annual rate, and effective annual rate are different conventions. Related terms: compounding frequency, continuous compounding, discounting, annualization Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Effective Annual Rate: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log Transform for Skewed Features

BEGINNER

ONE-SENTENCE MEANING

A log transform compresses large positive values and often makes right-skewed variables easier to model. Memory jogger: Log squeezes the right tail.

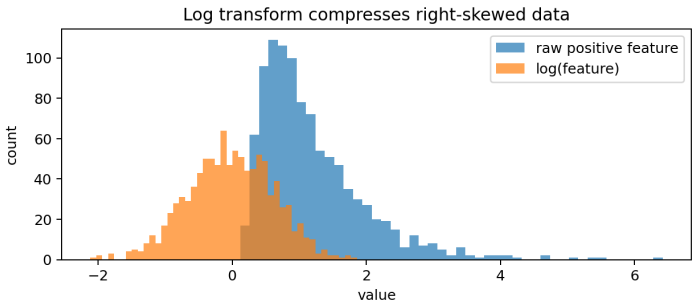
WHY IT MATTERS IN QUANT RESEARCH

Market cap, volume, turnover, spreads, and dollar values can have extreme skew that destabilizes models.

SIMPLE MARKET/TRADING EXAMPLE

Using log(volume) can reduce the dominance of mega-volume days in a feature set.

RAW VS LOG-TRANSFORMED DISTRIBUTION



The log transform compresses extreme positive values into a more compact scale.

FORMULA / INTUITION

```
x_log = log(x), for x > 0
x_log1p = log(1 + x), for x >= 0
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use np.log for strictly positive variables and np.log1p when zeros are valid. Store transform metadata.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Feature engineering, robust model inputs, cross-sectional ranking, and outlier management.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Logging a feature changes coefficient interpretation. It does not automatically fix bad data. Related terms: feature scaling, skew, normalization, log1p Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log Transform for Skewed Features: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Zeros, Negatives, and Logs

BEGINNER

ONE-SENTENCE MEANING

Standard logarithms require positive inputs, so zero and negative values need explicit handling. Memory jogger: No positive input, no normal log.

WHY IT MATTERS IN QUANT RESEARCH

Bad log handling can silently create NaNs, infinities, dropped rows, or biased training data.

SIMPLE MARKET/TRADING EXAMPLE

Volume can be zero, spreads can be invalid, and returns can hit -100 percent. Each case needs a rule before logging.

FORMULA / INTUITION

```
log(x) is defined for x > 0
log1p(x) is defined for x > -1
```

LOG INPUT HANDLING

Input issue	Possible rule	Risk
zero values	log1p or small floor	floor may distort scale
negative values	signed transform or no log	interpretation changes
missing values	impute or mask	may bias training
return <= -1	flag as invalid/total loss	breaks log1p

PYTHON / SYSTEM IMPLEMENTATION IDEA

Create a validation function before every log transform. Count invalid values and write them to a data-quality report.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Data cleaning, feature engineering, training pipelines, and production monitoring.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Adding a tiny constant is not neutral. It can change rankings near zero. Related terms: log transform, log1p, data validation, missing data
Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Zeros, Negatives, and Logs: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Numerical Stability

BEGINNER

ONE-SENTENCE MEANING

Numerical stability means writing calculations so tiny or huge values do not create avoidable precision errors. Memory jogger: Stable formulas preserve meaning under machine precision.

WHY IT MATTERS IN QUANT RESEARCH

Long backtests, tiny returns, likelihoods, and exponential transforms can magnify floating-point issues.

SIMPLE MARKET/TRADING EXAMPLE

Use log1p(r) instead of log(1+r) for tiny returns; use expm1(x) instead of exp(x)-1 for tiny log returns.

FORMULA / INTUITION

```
stable log return: ell = log1p(r)
stable simple return: r = expm1(ell)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Prefer numpy stable functions, validate finite results with np.isfinite, and monitor NaN/inf counts after transformations.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Production research infrastructure, feature pipelines, backtest engine, and unit tests.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Numerical stability is not theoretical decoration. It prevents small bugs from accumulating across millions of rows. Related terms: log1p, expm1, floating point, data validation Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Numerical Stability: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Feature Stability Map

BEGINNER

ONE-SENTENCE MEANING

A feature stability map checks whether transformed features remain reliable across time periods or regimes. Memory jogger: A good transform should not only look good in one sample.

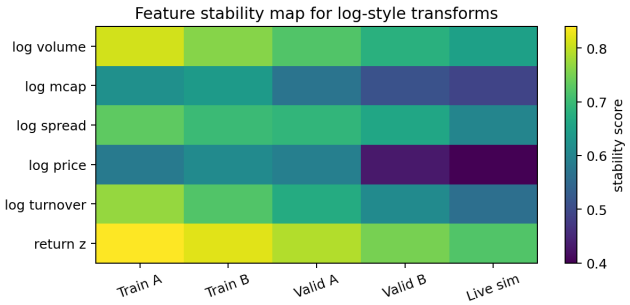
WHY IT MATTERS IN QUANT RESEARCH

Log transforms can improve stability, but they still need validation across train, validation, and live-like windows.

SIMPLE MARKET/TRADING EXAMPLE

A log volume feature may work in quiet markets but decay during high-volatility regimes if liquidity behavior changes.

FEATURE STABILITY HEATMAP



Rows are transformed features and columns are research periods.

FORMULA / INTUITION

```
stability_score = similarity(feature_behavior_train, feature_behavior_test)
common checks: rank correlation, missing rate, coefficient sign, IC consistency
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Build a report that compares missing rate, distribution shape, rank IC, and model coefficient sign across windows.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Feature validation, model governance, walk-forward testing, and live monitoring.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A mathematically valid transform can still be a poor live feature if its behavior is unstable. Related terms: feature engineering, regime analysis, log transform, validation Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Feature Stability Map: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Parameter Sensitivity for Compounding

BEGINNER

ONE-SENTENCE MEANING

Parameter sensitivity checks how outputs change when compounding assumptions or transform parameters change. Memory jogger: Robust systems should not collapse under tiny parameter changes.

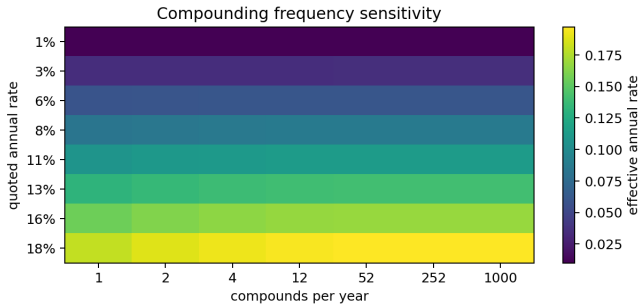
WHY IT MATTERS IN QUANT RESEARCH

Compounding frequency, smoothing alpha, half-life, and leverage all affect final performance.

SIMPLE MARKET/TRADING EXAMPLE

A strategy that only works under one exact EMA half-life is probably fragile.

COMPOUNDING PARAMETER HEATMAP



Effective return changes with quoted rate and compounding frequency.

FORMULA / INTUITION

```
sensitivity = change_in_metric / change_in_parameter
robustness check: evaluate grid of plausible parameters
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Run parameter sweeps and store metric surfaces. Look for broad stable regions, not one lucky peak.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Research validation, hyperparameter review, stress testing, and deployment readiness checks.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: The best in-sample parameter is not automatically the best production parameter. Related terms: heatmap, robustness, compounding frequency, overfitting Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Parameter Sensitivity for Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

Backtest Compounding Logic

BEGINNER

ONE-SENTENCE MEANING

Backtest compounding logic is the code path that converts strategy returns into an equity curve. Memory jogger: The equity curve is only as correct as the compounding engine.

WHY IT MATTERS IN QUANT RESEARCH

A backtest can have good signals but wrong results if returns are aggregated incorrectly.

SIMPLE MARKET/TRADING EXAMPLE

A daily strategy return column should usually become equity through (1+r).cumprod(), after costs and slippage.

FORMULA / INTUITION

```
net_return_t = gross_return_t - costs_t - slippage_t
equity_t = equity_{t-1} * (1 + net_return_t)
```

BACKTEST COMPOUNDING CHECKS

Check	Expected behavior	Bug symptom
returns finite	no NaN/inf after cost model	equity gaps or crashes
r > -1	growth factor nonnegative	negative/invalid equity
cost timing	cost before compounding	inflated performance
cumprod	multiplicative equity	wrong total return

PYTHON / SYSTEM IMPLEMENTATION IDEA

Write unit tests with small known return vectors. Example: [0.1, -0.1] should end at 0.99x before costs.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Backtest engine, performance metrics, audit layer, and live strategy simulation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not mix gross and net returns in the same cumulative equity curve. Related terms: growth factor, cumprod, costs, equity curve Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Backtest Compounding Logic: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Return Aggregation Failures

BEGINNER

ONE-SENTENCE MEANING

Return aggregation failures occur when returns are combined with the wrong arithmetic for their unit. Memory jogger: Know whether your return object adds or multiplies.

WHY IT MATTERS IN QUANT RESEARCH

Aggregation bugs can produce false strategy performance and are common in early research systems.

SIMPLE MARKET/TRADING EXAMPLE

Summing daily simple returns over a year can overstate or understate actual compounded performance.

FORMULA / INTUITION

```
simple returns over time: product(1+r_t)-1
log returns over time: sum(ell_t)
```

FAILURE-MODE TABLE

Mistake	Cause	Fix
cumsum simple returns	treating multipliers as additive	use cumprod(1+r)
cumprod log returns	treating logs as growth factors	use exp(cumsum(log_r))
wrong annualization	multiplying simple daily returns	compound or use log returns
forgot costs	gross returns only	deduct costs before compounding

PYTHON / SYSTEM IMPLEMENTATION IDEA

Create a returns schema: simple_return, log_return, growth_factor, equity_level. Make functions reject ambiguous columns.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Data contracts, backtest QA, feature validation, and metrics publishing.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A high Sharpe from incorrectly aggregated returns is not a valid research result. Related terms: backtest accounting, simple return, log return, annualization Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Return Aggregation Failures: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Strategy Evaluation with Log Growth

BEGINNER

ONE-SENTENCE MEANING

Log growth evaluates a strategy by the additive growth rate of its compounded wealth. Memory jogger: Mean log return measures compound-growth tendency.

WHY IT MATTERS IN QUANT RESEARCH

Mean log return can reveal path quality that arithmetic average return hides.

SIMPLE MARKET/TRADING EXAMPLE

Two strategies may have the same average simple return, but the one with higher mean log return compounds better.

FORMULA / INTUITION

```
mean_log_growth = mean(log(1 + r_t))
annual_log_growth = periods_per_year * mean_log_growth
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Report mean log return alongside CAGR, volatility, drawdown, and turnover. Use log growth for aggregation, not as the sole decision metric.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Strategy ranking, research dashboards, risk-adjusted evaluation, and model comparison.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Mean log growth does not replace risk controls or capacity analysis. Related terms: Kelly criterion, CAGR, volatility drag, log return Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Strategy Evaluation with Log Growth: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Kelly Growth Intuition

BEGINNER

ONE-SENTENCE MEANING

Kelly-style thinking maximizes expected log growth, balancing return against risk of compounding losses. Memory jogger: Bet size should respect the path of wealth.

WHY IT MATTERS IN QUANT RESEARCH

It explains why overbetting can reduce long-run growth even with positive edge.

SIMPLE MARKET/TRADING EXAMPLE

A strategy with positive expected return can still destroy capital if position sizing creates severe drawdowns.

FORMULA / INTUITION

```
objective idea: maximize E[log(W_{t+1}/W_t)]
wealth update: W_{t+1} = W_t * (1 + f*r_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use fractional Kelly or risk-constrained sizing in practical systems. Estimate uncertainty conservatively.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Position sizing, risk budgeting, leverage limits, and long-run growth analysis.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Kelly is highly sensitive to estimated edge and distribution assumptions. It is not a license to use maximum leverage. Related terms: expected log growth, leverage, volatility drag, position sizing Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Kelly Growth Intuition: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Compounding and Transaction Costs

BEGINNER

ONE-SENTENCE MEANING

Transaction costs reduce the growth factor in each period before compounding, which can heavily affect long-run wealth. Memory jogger: Small costs repeat into large performance drag.

WHY IT MATTERS IN QUANT RESEARCH

High-turnover strategies can lose their edge once costs compound through time.

SIMPLE MARKET/TRADING EXAMPLE

A strategy making many small trades can show strong gross returns and weak or negative net compounded returns.

FORMULA / INTUITION

```
net_return_t = gross_return_t - cost_t
net_equity = start * product(1 + net_return_t)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Deduct costs at the same frequency as trades or rebalances. Model slippage, fees, spreads, borrow, and financing separately when possible.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Backtest realism, execution modeling, capacity analysis, and strategy triage.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Do not subtract one annual cost estimate from final return if costs occur throughout the path. Related terms: turnover, slippage, net return, growth factor Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Compounding and Transaction Costs: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log-Likelihood Connection

BEGINNER

ONE-SENTENCE MEANING

Log-likelihood uses logs to turn products of probabilities into sums, making estimation tractable. Memory jogger: Logs turn probability products into score sums.

WHY IT MATTERS IN QUANT RESEARCH

Many statistical models, including regression and probabilistic forecasting, optimize log-likelihoods.

SIMPLE MARKET/TRADING EXAMPLE

A model evaluating many daily outcomes multiplies likelihood terms; logs convert that into a sum of log probabilities.

FORMULA / INTUITION

```
likelihood = product_i p(data_i | theta)
log_likelihood = sum_i log(p(data_i | theta))
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use library loss functions that operate in log space when probabilities are tiny. Avoid multiplying many small probabilities directly.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Model training, probabilistic signals, calibration, and statistical validation.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Log-likelihood is not a return, but the same log-product-to-sum identity is doing the work. Related terms: additivity of logs, maximum likelihood, probability, optimization Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log-Likelihood Connection: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Log Scale Charts

BEGINNER

ONE-SENTENCE MEANING

A log scale displays equal percentage moves as equal vertical distances. Memory jogger: Log charts compare proportional moves, not dollar moves.

WHY IT MATTERS IN QUANT RESEARCH

Log scales make long-term price charts and multi-asset comparisons easier to interpret.

SIMPLE MARKET/TRADING EXAMPLE

A move from 10 to 20 and a move from 100 to 200 are both 100 percent gains; a log chart gives them equal vertical size.

FORMULA / INTUITION

```
log-axis position proportional to log(price)
vertical difference = log(P_2) - log(P_1)
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Use log scales for long-horizon price visualizations, but label the axis clearly.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Visualization layer, research notebooks, dashboards, and regime analysis.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: A log chart can make large dollar moves look smaller because it emphasizes percentage change. Related terms: log price, exponential growth, proportional change, visualization Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Log Scale Charts: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Regime Effects on Compounding

BEGINNER

ONE-SENTENCE MEANING

Different market regimes change the distribution and sequence of returns, which changes compounded growth. Memory jogger: The same model can compound differently in a different regime.

WHY IT MATTERS IN QUANT RESEARCH

A strategy may look strong in trending regimes and degrade in choppy or crisis regimes.

SIMPLE MARKET/TRADING EXAMPLE

Momentum may compound well in persistent trends but suffer in violent reversals.

FORMULA / INTUITION

```
regime_return_profile = distribution(r_t | regime)
equity = product over regime-specific growth factors
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Segment backtests by volatility, trend, liquidity, and macro regimes. Compare CAGR, drawdown, mean log return, and turnover by segment.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Robustness testing, walk-forward validation, risk controls, and live monitoring.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Full-sample CAGR can hide regime-specific failure. Related terms: regime analysis, feature stability, drawdown, volatility drag Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Regime Effects on Compounding: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Decision Tree for Return Math

BEGINNER

ONE-SENTENCE MEANING

A return-math decision tree selects the correct formula based on whether the object is a level, simple return, log return, or growth factor. Memory jogger: Identify the unit before choosing the operation.

WHY IT MATTERS IN QUANT RESEARCH

This prevents most compounding and log-return errors in research code.

SIMPLE MARKET/TRADING EXAMPLE

If you have prices, use percent change or log diff. If you have simple returns, use cumprod. If you have log returns, use cumsum then exp.

FORMULA / INTUITION

```
prices -> log(price).diff()
simple returns -> (1+r).cumprod()
log returns -> exp(log_returns.cumsum())
```

METHOD SELECTION TABLE

You have	Need	Use
price levels	simple returns	pct_change
price levels	log returns	log(price).diff
simple returns	equity curve	cumprod(1+r)
log returns	equity curve	exp(cumsum(log_r))

PYTHON / SYSTEM IMPLEMENTATION IDEA

Write wrapper functions with explicit input_type arguments. Reject ambiguous names like returns unless schema metadata is present.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Reusable data utilities, backtest engine, and validation checks before metrics are computed.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Most mistakes happen before modeling, during basic unit conversion. Related terms: schema, return aggregation, log1p, cumprod Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Decision Tree for Return Math: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Core Formula Sheet

BEGINNER

ONE-SENTENCE MEANING

A formula sheet compresses the most reusable exponent, log, and compounding relationships into one lookup page. Memory jogger: Use this when debugging calculations.

WHY IT MATTERS IN QUANT RESEARCH

These formulas are the minimal toolkit for return conversion, growth, annualization, and feature transforms.

SIMPLE MARKET/TRADING EXAMPLE

If a calculation looks wrong, first check whether you used the formula for the correct unit.

FORMULA / INTUITION

```
growth factor: g_t = 1 + r_t
equity: V_t = V_0 * product(1+r_t)
log return: ell_t = log(P_t/P_{t-1})
simple from log: r_t = exp(ell_t)-1
CAGR: (V_T/V_0)^(1/years)-1
half-life: log(2)/lambda
```

PYTHON / SYSTEM IMPLEMENTATION IDEA

Turn these formulas into tested utility functions. Never duplicate formula code in many notebooks.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Reference layer, test suite, research notebooks, and production feature library.

CONFUSIONS, RELATED TERMS, QUICK RECALL

Confusions to avoid: Memorizing formulas is less important than knowing which object each formula expects. Related terms: all core concepts in this manual Quick recall: What unit is this object in: price, return, log return, growth factor, or parameter?

VALIDATION CHECKLIST

Validation checklist for Core Formula Sheet: identify the input unit, check the valid domain, run one tiny numeric example, confirm whether outputs add or multiply, and store the result with an explicit column name.

PRACTICAL PITFALL

Practical pitfall: most errors do not come from the formula itself. They come from applying the formula to the wrong object, using percent units instead of decimal units, or forgetting the conversion back to growth-factor space.

Formula Review Sheet

REVIEW

PURPOSE

Use this page as a compact recall sheet after reading the concept cards. Every formula should be tied to a data object in your system.

CORE FORMULA COMPARISON

Task	Formula	System object
Price to simple return	r_t = P_t/P_{t-1} - 1	simple_return column
Price to log return	ell_t = log(P_t/P_{t-1})	log_return column
Simple to log	ell_t = log1p(r_t)	conversion utility
Log to simple	r_t = expm1(ell_t)	conversion utility
Compound equity	V_t = V_0 * product(1+r_t)	equity curve
Log equity	V_t = V_0 * exp(sum ell_t)	equity reconstruction
CAGR	(V_T/V_0)^(1/years)-1	performance report
Half-life	log(2)/lambda	signal decay report

HIGH-VALUE IDENTITIES

```
log(a*b) = log(a) + log(b)
log(a/b) = log(a) - log(b)
exp(log(x)) = x for x > 0
log(1+r) approx r for small r
geometric growth approx arithmetic mean - variance/2
```

These are the formulas that explain why logs, compounding, and volatility drag are linked.

QUICK RECALL QUESTIONS

1. Which object do you cumprod? 2. Which object do you cumsum? 3. Why does a +50 percent move and a -50 percent move not cancel? 4. When should you use log1p? 5. What does half-life measure?

Diagnostics and Failure Modes

REVIEW

PURPOSE

Use this page as a debugging checklist for return math, log transforms, and compounding assumptions. Most early quant bugs are unit bugs, not advanced math bugs.

DIAGNOSTIC TABLE

Symptom	Likely cause	Fix
Equity curve too smooth	returns may be averaged before compounding	compound period returns directly
Equity curve explodes	percent values used as decimals, e.g. 2 instead of 0.02	standardize decimal return units
NaN log returns	zero, negative, or missing price inputs	validate prices before log diff
CAGR inconsistent with total return	wrong year count or simple annualization	use geometric formula with elapsed time
Great mean return, poor ending equity	volatility drag or clustered losses	inspect mean log return and drawdown
Different notebook results	duplicated formula logic	centralize utility functions and tests
Live feature drift	transform unstable across regimes	monitor feature stability by period

ASSUMPTION TABLE

Method	Assumption	Breaks when
log returns	positive price levels	price <= 0 or bad adjustments
CAGR	meaningful start/end values	cash flows distort equity
half-life	exponential decay structure	decay is regime-dependent
EMA	recent observations deserve more weight	old regimes still matter
annualization	periods are comparable	irregular timestamps or missing data

SYSTEM-DESIGN CONNECTION

A serious research system should store metadata for every transformed column: source column, transform formula, valid input range, units, frequency, and whether the value is additive or multiplicative.

Python Implementation Checklist

REVIEW

PURPOSE

This is the implementation-oriented version of the manual. The goal is to translate math into repeatable, testable research functions.

FUNCTION CHECKLIST

Utility	Expected implementation	Test case
simple_returns(price)	price.pct_change()	100 -> 105 gives 0.05
log_returns(price)	np.log(price).diff()	100 -> 105 gives log(1.05)
to_log_return(r)	np.log1p(r)	r=0 returns 0
to_simple_return(l)	np.expm1(l)	l=0 returns 0
equity_from_returns(r)	start*(1+r).cumprod()	[.1,-.1] gives .99x
cagr(equity, years)	(end/start)**(1/years)-1	100->121 in 2 years gives 10%
half_life(lambda)	np.log(2)/lambda	lambda=log(2)/10 gives 10

MINIMAL CODE PATTERN

```
growth = 1 + returns
equity = start_value * growth.cumprod()
log_r = np.log1p(returns)
equity_from_log = start_value * np.exp(log_r.cumsum())
```

These two equity curves should match within tolerance when returns are valid.

VALIDATION RULES

Validate decimal units, positive prices, returns greater than -100 percent, finite outputs, expected frequency, and no lookahead in weights. Fail fast instead of silently producing a polished wrong chart.

Index

REFERENCE

INDEX USAGE

This index groups the most important lookup terms by how they appear in a quant research system.

RETURN MATH

simple return, log return, growth factor, cumulative product, cumulative log return, arithmetic vs log returns, annualization

COMPOUNDING

compound growth, CAGR, geometric return, continuous compounding, compounding frequency, effective annual rate, portfolio compounding

LOGS AND EXPONENTS

exponents, exponent laws, exp, e, natural log, log base change, inverse relationship, log scale charts

RISK/PATH EFFECTS

volatility drag, drawdown in log scale, leverage, transaction costs, regime effects, Kelly growth intuition

DECAY AND SMOOTHING

exponential decay, half-life, EMA, exponential weighting, mean reversion as decay

FEATURE ENGINEERING

log transform, log1p, expm1, zeros and negatives, numerical stability, feature stability map

SYSTEM IMPLEMENTATION

schema, validation, backtest compounding logic, return aggregation failures, Python utilities

FINAL MEMORY ANCHOR

Exponents multiply growth through time. Logs turn multiplicative growth into additive math. Compounding is the actual path your capital follows. Keep these units separate and your research system becomes much easier to debug.